

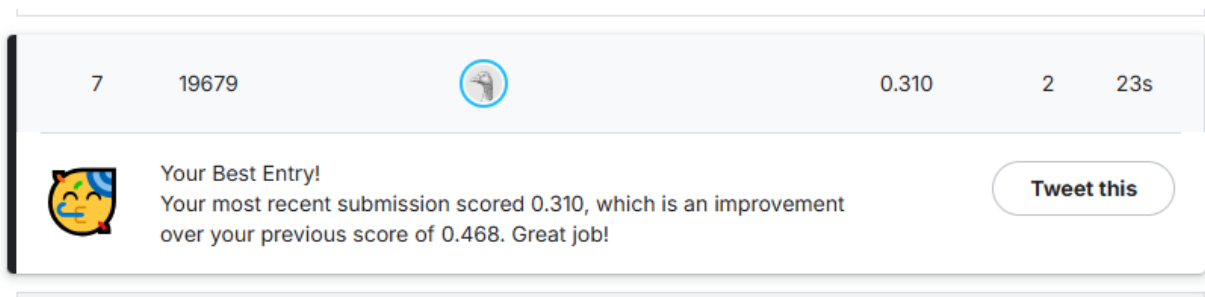
CS 534 HW2

0.

These steps are needed to be able to keep everything consistent and to minimize the chance for error and reduce the noise. Each step makes it so that every base case is covered and nothing unexpected happens when an unexpected token occurs. This also prevents words like “they’re” being treated differently based on how the user typed it. The model could think “they’re” is positive and “they” and “are” as either both positive, negative, or one of each. This would not be ideal.

1.

- 1.1. This class can support those operations because the class has the needed functions that can do those operations.
- 1.2. The train function teaches the model by going through the training data several times and adjusting the weights whenever it gets a sentence wrong. After each round, it checks how well the model performs using the test function and saves the version that makes the fewest mistakes. The test function measures the model’s accuracy by seeing how many sentences it classifies incorrectly.
- 1.3. I added `v "<bias>" +=1` inside the `make_vector` function just outside of the for loop. This improved my dev error from 30.1% to 28.9%.
- 1.4. My best dev rate is 28.9%, and I'm currently ranked number 7 on the leaderboard. It has a public error rate of 31%



The screenshot shows a Kaggle leaderboard entry for a user with rank 7 and a score of 0.310. A notification banner at the bottom of the entry reads: "Your Best Entry! Your most recent submission scored 0.310, which is an improvement over your previous score of 0.468. Great job!" with a "Tweet this" button.

Rank	Score	Public Error Rate	Private Error Rate	Time
7	0.310	31%	28.9%	23s

- 1.5. The reason you need a bias with a 50/50 split is because this bias allows the decision boundary to shift away from the origin. Without a bias, the classifier is forced to place the hyperplane through the origin, which may not align well with the actual distribution of the data. The bias gives the model more flexibility to separate the positive and negative examples more accurately (even when the dataset is a perfect 50/50 split).

2.

- 2.1. Yes, averaging did in fact improve the dev error rates. Before averaging, The best dev rate was 28.9%, and after, it was 26.3%. Yes, the dev error rates did become more stable. There is not as much variance between each epoch compared to the first version.
- 2.2. No, smart averaging did not slow down the training at all.
- 2.3. Most of the words that are in the 20 top positive and negative make sense, but there are a few outliers that don't make sense to me. The words from the positive list that don't make sense would be: french, provides, pulls, skin, open, and dots. For the negative words that don't make sense would be, too routine, instead, tv, and guy.

```
Top 20 words
1. engrossing 11.41575
2. rare 10.559785714285715
3. french 10.346660714285715
4. unexpected 9.989678571428572
5. provides 9.784089285714286
6. triumph 9.724589285714284
7. cinema 9.630749999999999
8. powerful 9.613357142857144
9. treat 9.52025
10. wonderful 9.518214285714286
11. pulls 9.494160714285714
12. skin 9.456535714285714
13. open 9.360339285714286
14. delightful 9.140392857142857
15. culture 9.129803571428571
16. refreshingly 9.129232142857143
17. dots 9.010339285714284
18. beautiful 8.992553571428571
19. speaks 8.679517857142857
20. unique 8.423482142857143
```

```
Top 20 words
1. cliché -8.75842857142857
2. neither -8.819857142857142
3. suffers -8.821767857142857
4. unless -8.980107142857143
5. worst -9.045857142857143
6. flat -9.142767857142857
7. guy -9.160964285714286
8. attempts -9.344553571428571
9. bad -9.443714285714286
10. ill -9.680160714285714
11. tv -9.694714285714285
12. instead -9.895535714285714
13. problem -10.052464285714287
14. routine -10.063375
15. badly -10.20330357142857
16. too -10.351303571428572
17. fails -10.417482142857143
18. generic -10.978785714285713
19. dull -12.793767857142857
20. boring -14.252821428571428
```

- 2.4.
- 2.4.1. Positive in dev and negative in my model**
- 2.4.1.1. 8004,"because the film deliberately lacks irony , it has a genuine dramatic impact it plays like a powerful 1957 drama we 've somehow never seen before",+
- 2.4.1.2. 8031,"writer director david jacobson and his star , jeremy renner , have made a remarkable film that explores the monster 's psychology not in order to excuse him but rather to demonstrate that his pathology evolved from human impulses that grew hideously twisted",+

- 2.4.1.3. 8037,"you 've already seen city by the sea under a variety of titles , but it 's worth yet another visit",+
- 2.4.1.4. 8058,"devotees of star trek ii the wrath of khan will feel a nagging sense of deja vu , and the grandeur of the best next generation episodes is lacking",+
- 2.4.1.5. 8067,"too much of storytelling moves away from solondz 's social critique , casting its audience as that of intellectual lector in contemplation of the auteur 's professional injuries",+

2.4.2. Negative in dev and positive in my model

- 2.4.2.1. 8001,an atonal estrogen opera that demonizes feminism while gifting the most sympathetic male of the piece with a nice vomit bath at his wedding,-
- 2.4.2.2. 8011,two tedious acts light on great scares and a good surprise ending,-
- 2.4.2.3. 8026,"it 's a long way from orwell 's dark , intelligent warning cry 1984 to the empty stud knockabout of equilibrium , and what once was conviction is now affectation",-
- 2.4.2.4. 8030,"oversexed , at times overwrought comedy drama that offers little insight into the experience of being forty , female and single",-
- 2.4.2.5. 8032,"unlike trey parker , sandler does n't understand that the idea of exploiting molestation for laughs is funny , not actually exploiting it yourself",-

2.4.3. Observations:

- 2.4.3.1. When viewing the positive in dev, but negative in my model I do see why it would think that. Going through the examples I see words like 'lacks', 'never', 'hideously', 'nagging', and 'injuries'. These example sentences do have positive aspects to them, but i'm guessing that the 'negative' words in them might be weighted more and thus marking it as a negative sentence instead of a positive. The same can be said about the negative in dev, but positive in my model. Words such as 'gifting', 'sympathetic', 'nice', 'great', 'good', 'laughs', and 'funny' are examples of positive vocabulary. But in the context of these sentences, they are in fact negative and not positive. So my main take away would be that 'positive' and 'negative' words have varying degrees of weight attached to them that will sway the sentence one way or the other causing the wrong interpretation.

2.5. My ranking is 6, with a 26.3% dev error rate

6
19679

0.268
6
1d



Your Best Entry!

Your most recent submission scored 0.268, which is an improvement over your previous score of 0.310. Great job!

Tweet this

3.

- 3.1. Yes, it did slightly help improve the dev error rate from 26.3% to 25.9%.
- 3.2. The model shrank quite a bit. It went from 15,806 to 8,425. Yes, removing these one off words and shrinking the set did help with overfitting.
- 3.3. Yes, the update did change and got slightly higher. Before the pruning, epoch 10 had a 6.9% update and after had a 10.6%. This makes sense since the model is needing to update its weights more frequently due to having much less words to work with. So it needs to update the weights more frequently.
- 3.4. It slightly increased the training time, but not by much. When I ran it a few times, it varied by only .1 or .2 seconds.
- 3.5. No, it did not increase the dev error rate, it made it slightly worse. It went from being 25.9% pruning one occurrence word to 26.6% when pruning 2 occurrence words.
- 3.6. The best error rate is 25.9%. My rank on the leaderboard remained at 7th place with my model that I submitted in part 2 still being better then this model I just submitted. This new submission had a public score of 27.4%

7

19679



0.268

7

32s



Your Best Entry!

Your most recent submission scored 0.274, which is not an improvement of your previous score. Keep trying!

4.

- 4.1. I used the k-nn algorithm to try and make it better. I implemented a word pruning function to try and get it to prune words to make it more efficient. But, I found out that pruning 1 or 2 occurrence words actually made the dev error worse. So I stuck with no pruning. The other adjustment I used was to have a cosine metric when trying to find the best k value. The other adaptation I used was TF-IDF vectorization. I used it to convert sentences into numeric vectors that k-nn could process. This one seemed to have the biggest impact and best outcome for speed and best error rate.
- 4.2. The best dev error rate was 23.50% and had a user time of 15.365 seconds. The best k value found was at k = 27.
- 4.3. My public score was .250 and I was ranked 26th.

26

19679



0.250

9

26s



Your Best Entry!

Your most recent submission scored 0.250, which is the same as your previous score. Keep trying!

- 4.4. I learned a new form of machine learning. I came into the class with no ML exposure and so far I've learned k-nn and perceptron. I do think that this one is similar in complexity to k-nn. They both seem easy enough to understand and implement. But I do think perceptron is a bit more simplistic. I learned a new way weights can be added

for my program to learn from. It was really cool to be able to learn something new like this to get it to work not only from a new method, but from one that I've already tried before. It's interesting to see how these two algorithms differ but are able to be used to solve the same problem.

5.

5.1. My best dev rate that I was able to achieve was 23.50% with k-nn.

5.2. My best rank was 40th on the leaderboard with a public score of .250.

6.

6.1. I spent about 15 hours on this assignment.

6.2. I'd rank this assignment on the easy to moderate scale

6.3. I worked on this assignment alone

6.4. I feel like I have about 85-90% understanding of the material. I'd say the math that is involved with this is a bit over my head, but I feel like I grasp the concepts well enough to have an understanding of what is happening in the background.

6.5. I enjoyed this assignment a lot. It's really rewarding to see all of this work pay off and to have a machine learning application.